Deterministic DAG Scheduler with Greedy Fusion and Tile-Shape Co-optimization

MLSys 2026 Competition · Track A: Systems Engineering · Bharath Velamala

OVERVIEW

We present a purely deterministic scheduler for memory-constrained DAG workloads written in Rust. The central design principle separates the two key decisions cleanly: a **greedy fusion pass** decides which operations to group into subgraphs, while an **exhaustive granularity search** selects the tile shape `[w, h, k]` that minimizes roofline latency subject to fast-memory capacity constraints. A post-processing **merge pass** then corrects look-ahead blindness in the greedy step by collapsing consecutive subgraph pairs whenever fusion reduces total latency. All constraint checking—working-set validation, split-K latency modeling, and zig-zag traversal—is handled analytically with no heuristics.

Greedy / search (no LLM) Deterministic (analytical)

ALGORITHM

Phase 1 — Tensor Role Analysis. For each candidate subgraph a `SubgraphTensors` struct is computed in a single pass over the operator list. It classifies every boundary input into one of three roles: MatMul LHS (kept fully resident across all k-steps, size `h × K_full`), MatMul RHS (streamed one strip at a time, size `k × w`), or Pointwise (one tile per spatial step, size `h × w`). This shared analysis is consumed by both the working-set and latency functions, eliminating duplicated classification loops.

Phase 2 — Greedy Subgraph Grouping. Ops are visited in topological order. For each unscheduled op we build a candidate group by repeatedly evaluating all schedulable consumers. A consumer c is added only if:

- All of c's producers are either already scheduled or in the current group
- `latency(group ∪ {c}) < latency(group) + latency({c})`

Phase 3 — Convergent Merge Pass. The greedy pass cannot look past locally suboptimal intermediate states. Consider fusing ops 0–3 into group A and ops 4–7 into group B: adding op 4 to A produces a larger intermediate tensor that appears more expensive, so the greedy pass stops—even though fusing all eight ops is optimal (the LHS is loaded once per output row rather than once per group per row). The merge pass corrects this: for every consecutive pair (A, B) it computes `latency(A ∪ B)` and replaces the pair if the merged cost is lower. This repeats until no further improvement is found.

Phase 4 — Granularity Search. For each finalized subgraph the scheduler exhaustively evaluates all `[w, h, k]` combinations drawn from multiples of `[native_w, native_h]`, sub-native powers-of-two (for tight-memory cases), and power-of-two subdivisions of `K_full`. Any combination whose peak working set exceeds fast-memory capacity is pruned. Among feasible candidates both raster and zig-zag traversal are evaluated; the one minimizing total roofline latency is selected.

Phase 5 — Tensor Retention. After each subgraph, output tensors consumed by the immediately following subgraph are retained in fast memory if their full size fits within `C - WS(next)`. Retention eliminates both the write-back and the future reload of that tensor.

LATENCY AND WORKING-SET MODELS

Each execution step is governed by a roofline model: `step_latency = max(compute_per_kstep, mem_bytes / bandwidth)`. For a subgraph with `k_steps = ⌈K_full / k⌉` the three step types carry different memory footprints:

- **Step 1:** load all non-resident LHS slices (`h × K_full` each) + all RHS strips (`k × w` each)
- **Middle steps:** stream RHS strips only — LHS is held resident
- **Last step:** stream RHS strips + write output tile (`h × w`) + load Pointwise inputs (`h × w` each)

When `tiles_w > 1`, zig-zag (snake) traversal enables intra-subgraph data reuse: LHS is loaded once at the first column tile and reused across all subsequent tiles in that row. For `k_steps = 1`, the last RHS tile of the previous row is also reused at the start of the next row. The exact zig-zag latency is: $L = L_{full} + (tiles_h - 1) \cdot L_{row} + tiles_h \cdot (tiles_w - 1) \cdot L_{col}$.

A critical correctness requirement for the working-set model: *every* non-ephemeral LHS tensor must be counted, not just the first. In a subgraph formed by fusing N independent MatMul chains, each chain contributes a fully-resident LHS. Counting only one underestimates working-set size, admits infeasible schedules, and causes runtime OOM errors. This is enforced uniformly across the granularity search, greedy pass, and merge pass.

RESULTS ON RELEASED BENCHMARKS

The baseline is a deterministic one-op-per-subgraph schedule with the algorithmically optimal granularity per step—the conservative lower bound for each problem.

Problem	Ops	Fast Mem	Baseline Latency	Scheduler Latency	Subgraphs	Speedup
mlsys-2026-1	5	60 K	340,787	340,787	1	1.00×
mlsys-2026-5	19	30 K	459,084	294,737	1	**1.56×**
mlsys-2026-9	32	250 K	6,118,638	5,558,430	1	**1.10×**
mlsys-2026-13	63	600 K	—	681,574	17	—
mlsys-2026-17	103	500 K	—	3,195,200	56	—

The scheduler's largest gains arise from the merge pass. On **mlsys-2026-5** (19 ops, 30 KB fast memory) the pass collapses all 19 ops into a single subgraph, achieving a 1.56× reduction by eliminating redundant slow-memory round trips for intermediate tensors. On **mlsys-2026-9** (32 MatMul-heavy ops on 1024×1024 tensors) the same pass merges all 32 ops into one subgraph; the single non-ephemeral LHS is loaded once per output row rather than once per original group, yielding a further 9% reduction over greedy-only. On memory-tight problems (**mlsys-2026-13**, 16 independent chains), each chain's LHS alone occupies 87% of fast memory, making cross-chain fusion infeasible; the scheduler correctly identifies this and produces 17 valid subgraphs with no OOM errors.

SUBMISSION DETAILS

Entry point (`mlsys`): accepts two positional arguments — `mlsys <input.json> <output.json>`. The binary is statically linked against musl libc and requires no runtime dependencies. All five benchmarks complete within 9 seconds on commodity hardware, well inside the contest time limits.

Core modules (`source/src/`): `scheduler.rs` — greedy grouping, merge pass, granularity search, zig-zag latency model, and tensor retention (~860 lines, no unsafe code); `problem.rs` — JSON deserialization of the problem spec; `solution.rs` — JSON serialization of the output schedule; `main.rs` — CLI entry point.

Module	Responsibility
`SubgraphTensors::analyze`	Single-pass LHS / RHS / Pointwise classification shared by WS and latency
`compute_working_set`	Peak fast-memory usage for a given `[w, h, k]`; rejects infeasible candidates
`compute_latency`	Exact split-K roofline with zig-zag traversal; asymmetric per-step costs
`find_best_granularity`	Exhaustive `[w, h, k]` search; emergency sub-native fallback
`scheduler_solution`	Greedy grouping → convergent merge pass → granularity assignment → retention

Dependencies: Rust standard library only; `anyhow` for error propagation; `serde` + `serde_json` for I/O. Runtime: Rust 1.80, target `x86_64-unknown-linux-musl`.